

EDITORA AGÊNTICA

# Checklist Mestre: IA Agêntica Desbloqueada

O checklist completo de 40 etapas para IA Agêntica Desbloqueada

NÍVEL INTERMEDIÁRIO

**Heverton Eduardo Peres**

Um recorte de ia-agentica-desbloqueada

# O checklist

---

São **40 verificações** distribuídas em 20 etapas. Marque cada uma antes de avançar — a ordem importa.

## Etapa 1 — O que é IA Agêntica (e o que ela não é)

*Definir IA agêntica com precisão, diferenciar de chatbots e automação tradicional, e mostrar o panorama de adoção em 2026.*

- ☐ ☐ Tratar o agente como um chatbot melhor\*\*: conectar um LLM a um prompt mais longo não cria um agente. Sem loop, ferramentas e estado, você tem um conversador eloquente
- ☐ ☐ Autonomia total desde o primeiro dia\*\*: comece com decisões de baixo impacto e supervisão humana; aumente a autonomia com evidência, não com entusiasmo

## Etapa 2 — O agent loop: perceber, raciocinar, agir

*Explicar o ciclo fundamental perceive-reason-act e suas variações práticas.*

- ☐ ☐ Loop sem observação\*\*: o agente executa a ferramenta e descarta o resultado — o ciclo não fecha e o sistema vira um chatbot com truques
- ☐ ☐ Ferramentas com descrições vagas\*\*: “faz coisas com dados” gera escolhas erradas. A descrição é parte da engenharia

## Etapa 3 — Arquiteturas de agente: do simples ao multiagente

*Apresentar os padrões arquiteturais: agente único, roteador, orquestrador-operários, swarm.*

- ☐ ☐ Multiagente prematuro\*\*: custo multiplicado sem ganho de qualidade. Estime o custo por missão antes de orquestrar
- ☐ ☐ Orquestrador gargalo\*\*: todo o tráfego passa pelo central; se ele falha, tudo falha. Adicione fallback e fila

## Etapa 4 — Fundamentos científicos: ReAct, memória e planejamento

*Ancorar o campo na literatura: ReAct, surveys de agentes, benchmarks e teorias de planejamento.*

- ☐ ☐ Agente sem trilha\*\*: um ReAct sem registro de pensamentos é um sistema sem memória de si — impossível de depurar e de auditar
- ☐ ☐ Memória sem recuperação seletiva\*\*: despejar o acervo inteiro na janela de contexto degrada a qualidade e explode o custo

## Etapa 5 — Engenharia de contexto para agentes

*Projetar o contexto do agente: instruções, exemplos, recuperação e o fim do prompt solto.*

- ☐ ☐ Prompt único gigante\*\*: uma instrução de 3.000 tokens com tudo misturado. Separe instrução, regras, exemplos e recuperação em camadas
- ☐ ☐ Despejo de recuperação\*\*: colocar 20 documentos recuperados no contexto. Selecione por relevância — o orçamento é parte do design

## Etapa 6 — Memória: curto prazo, longo prazo e vetorial

*Implementar sistemas de memória multi-escopo para agentes persistentes.*

- ☐ ☐ Memória como despejo\*\*: persistir tudo e recuperar tudo. O acervo gigante sem seleção degrada a resposta — curadoria é parte da memória
- ☐ ☐ Sem memória episódica\*\*: o sistema nunca aprende com a própria operação — cada erro é a primeira vez

## Etapa 7 — Ferramentas e function calling: as mãos do agente

*Projetar agent-computer interfaces (ACI): schemas, descrições e a arte do tool use confiável.*

- ☐ ☐ Ferramenta como função sem contrato\*\*: nome sem verbo, sem descrição, sem docstring — o modelo não sabe quando usar e escolhe errado
- ☐ ☐ Execução sem validação\*\*: confiar na saída do modelo é o erro mais caro — argumentos inválidos executam ações erradas em sistemas reais

## Etapa 8 — Planejamento de tarefas e decomposição

*Ensinar o agente a planejar: decomposição hierárquica, reflexão, autocorreção e o fim dos loops infinitos.*

- ☐ ☐ Missão como passo único\*\*: “resolver o problema do cliente” sem decomposição é uma intenção, não um plano — sem critérios verificáveis no meio

- ☐ ☐ Plano sem verificação\*\*: passos executados sem conferir o critério de sucesso — o agente “conclui” missões que não terminou

## Etapa 9 — Escolhendo o framework: LangGraph, CrewAI e além

*Comparar os frameworks de agentes e escolher a base tecnológica do OrquestralA.*

- ☐ ☐ Framework por hype\*\*: escolher LangGraph porque “todo mundo usa” sem comparar com o código puro — o fluxo simples paga complexidade desnecessária
- ☐ ☐ Abstração sem entendimento\*\*: usar o framework sem entender o loop por baixo — quando o trace dá errado, não há como depurar (este livro construiu o entendimento antes do framework, de propósito)

## Etapa 10 — O núcleo do OrquestralA: o orquestrador

*Construir o orquestrador central: despacho de tarefas, estados, persistência e checkpoints.*

- ☐ ☐ Orquestrador que executa\*\*: o central faz o trabalho dos especialistas — vira um agente gigante, não um orquestrador
- ☐ ☐ Roteamento cego\*\*: delegar ao especialista errado multiplica o erro — regras + LLM + rastreio de roteamento

## Etapa 11 — Conectando ao mundo: MCP e APIs

*Integrar o agente a ferramentas e dados via Model Context Protocol e APIs externas.*

- ☐ ☐ MCP por moda\*\*: adotar MCP para uma integração interna simples — a API direta é mais leve. Decida por critérios, não por hype
- ☐ ☐ Token em código\*\*: credenciais no código-fonte vazam — variáveis de ambiente e cofres (Capítulo 17) são obrigatórios

## Etapa 12 — Sistemas multiagentes na prática

*Implementar colaboração entre agentes: supervisor, crítico, especialistas e comunicação A2A.*

- ☐ ☐ Multiagente por estética\*\*: “meu sistema tem 10 agentes” como objetivo — cada agente deve justificar o custo com benefício medido
- ☐ ☐ Sobreposição de papéis\*\*: dois agentes com o mesmo escopo confundem o roteamento e dobram o custo — escopo único por agente

## Etapa 13 — Avaliando agentes: evals e LLM-as-a-judge

*Construir a infraestrutura de avaliação: graders de código, de modelo e humanos.*

- ☐ ☐ Avaliar só a resposta final\*\*: o agente que erra a ferramenta mas escreve bem “passa” — os evals de agente avaliam o caminho, não só o destino
- ☐ ☐ Golden set que muda o tempo todo\*\*: sem conjunto fixo não há regressão — as mudanças entram sem saber se pioraram

## Etapa 14 — Segurança: prompt injection e tool poisoning

*Mapear as ameaças específicas de agentes e implementar defesas em profundidade.*

- ☐ ☐ Confiar no modelo\*\*: achar que o LLM “entende” a diferença entre dado e instrução sem marcação estrutural — ele não; a separação é sua responsabilidade
- ☐ ☐ Ferramenta sem política\*\*: expor ferramentas sem o permissor — qualquer chamada é possível, e o abuso é uma questão de quando

## Etapa 15 — Supervisão humana: human-in-the-loop

*Projetar os pontos de intervenção humana: limiares de confiança, aprovação síncrona e auditoria assíncrona.*

- ☐ ☐ Autonomia total\*\*: sem HITL, o sistema executa ações irreversíveis com base em modelo que erra — a falha mais previsível do mercado
- ☐ ☐ Supervisão de fachada\*\*: fila de aprovação que o humano carimba sem contexto — o pior dos mundos: custo da supervisão sem o benefício

## Etapa 16 — Observabilidade e custos de tokens

*Instrumentar o agente: traces, métricas, logs e controle do custo de inferência.*

- ☐ ☐ Logar sem estruturar\*\*: linhas de log soltas sem as quatro dimensões — impossível resumir, comparar e alertar
- ☐ ☐ Painel sem trilha\*\*: métricas agregadas sem o detalhe de cada missão — o painel diz que algo está errado, a trilha diz o quê

## Etapa 17 — Implantando o OrquestraIA em produção

*Levar o sistema para produção: LLM gateways, fallback, escalabilidade e CI/CD de agentes.*

- ☐ ☐ Sem gateway\*\*: chamadas diretas aos provedores espalhadas — sem fallback, sem cache, sem observação de custo
- ☐ ☐ Segredo no código\*\*: a chave no repositório é a primeira vulnerabilidade que um atacante procura — ambiente/cofre sempre

## **Etapa 18 — Casos de uso reais: suporte, vendas e análise**

*Aplicar o OrquestraIA a cenários reais: atendimento, prospecção e análise de dados.*

- ☐ ☐ Mesmo agente para todos os domínios\*\*: tratar suporte, vendas e análise com o mesmo desenho — cada domínio tem ênfase própria (rotas, autonomia, verificação)
- ☐ ☐ Suporte sem memória\*\*: atender sem lembrar o cliente — o CSAT de relacionamento exige memória entre sessões

## **Etapa 19 — Operação contínua: iteração, feedback e evolução**

*Operar o sistema no tempo: coleta de feedback, reavaliação e evolução sem reescrita.*

- ☐ ☐ Operar sem medir\*\*: o painel que ninguém lê ou as métricas que não existem — a operação vira opinião
- ☐ ☐ Erro sem lição\*\*: incidentes resolvidos e esquecidos — o erro repetido é a falha da operação, não do sistema

## **Etapa 20 — O engenheiro de sistemas agênticos**

*Consolidar a carreira e a mentalidade: o perfil do engenheiro que projeta, constrói e implanta autonomia.*

- ☐ ☐ Ficar na superfície\*\*: dominar prompts e demos sem o núcleo técnico — o mercado paga pela profundidade, não pela superfície
- ☐ ☐ Construir sem medir\*\*: sistemas sem evals e painel — protótipos, não produtos

# Próximo passo

Este material é um recorte de **IA Agêntica Desbloqueada**. A obra completa traz a teoria, os exemplos comentados e as referências.

**Quero o livro completo** — [https://pay.hotmart.com/XXXXX?utm\\_source=lead-magnet&utm\\_medium=pdf&utm\\_campaign=ia-agentica-desbloqueada&utm\\_content=checklist](https://pay.hotmart.com/XXXXX?utm_source=lead-magnet&utm_medium=pdf&utm_campaign=ia-agentica-desbloqueada&utm_content=checklist)